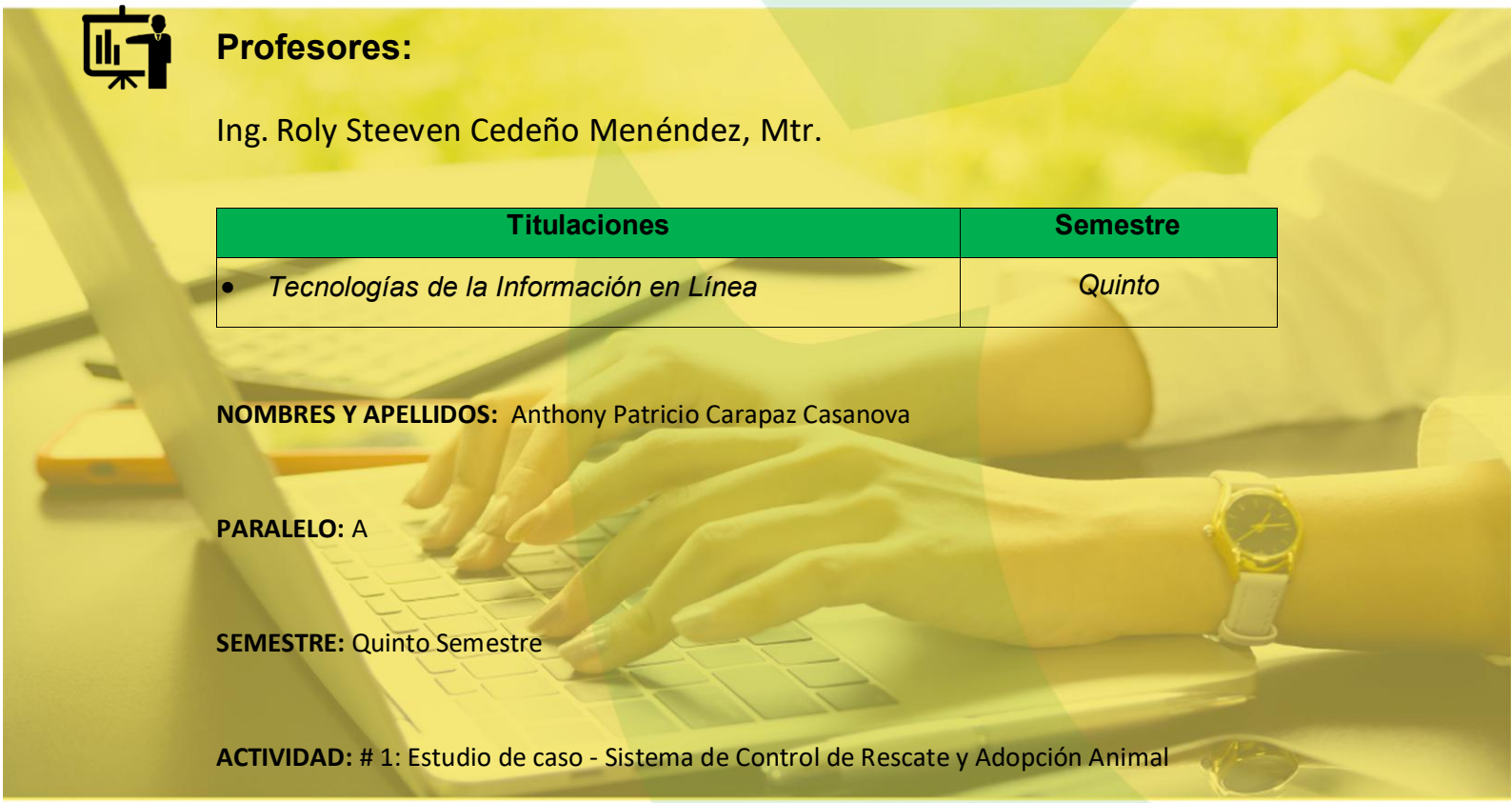
UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Anthony Patricio Carapaz Casanova

PARALELO: A

SEMESTRE: Quinto Semestre

ACTIVIDAD: # 1: Estudio de caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad ____

Actividad # 1 Estudio de caso - Sistema de Control de Rescate y Adopción Animal

INTRODUCCION. –

El rescate y la adopción de animales son actividades muy importantes, ya que permiten ayudar a muchos animales que necesitan cuidado, atención médica y un hogar donde puedan vivir seguros. Sin embargo, organizar toda la información de cada animal no siempre es fácil, especialmente cuando los datos se encuentran en diferentes documentos o se llevan de forma manual. Esto puede ocasionar confusiones y dificultar el seguimiento de cada caso.

Por eso, en este proyecto se propone diseñar un Sistema de Control de Rescate y Adopción Animal que permita llevar un mejor control de los animales rescatados, conocer su especie, registrar su estado de salud y organizar la participación de voluntarios y veterinarios. Además, el sistema permitirá dar seguimiento a cada animal desde el momento de su rescate hasta que esté listo para ser adoptado por una familia responsable.

Para desarrollar esta propuesta se utilizarán diagramas UML, principios SOLID y patrones de diseño, que ayudarán a organizar mejor la estructura del sistema. También se trabajará con GitHub y la metodología Gitflow para guardar los avances, controlar los cambios realizados y mantener el proyecto ordenado durante su desarrollo.

Con este proyecto se busca plantear una solución sencilla, organizada y fácil de comprender, que permita mejorar la gestión de la información y facilite el trabajo de las personas que ayudan en el rescate y cuidado de los animales. De esta manera, se espera contar con una base que pueda mejorarse en el futuro y que contribuya a que cada animal tenga la oportunidad de recibir la atención que necesita y encontrar un hogar adecuado.

OBJETIVOS

Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante diagramas UML, patrones de diseño de software, principios SOLID y el flujo de trabajo Gitflow, con el fin de establecer una solución organizada, mantenible y escalable.

OBJETIVOS ESPECÍFICOS

- *Identificar los actores, requerimientos funcionales y requerimientos no funcionales del sistema.*
- *Representar la estructura y el funcionamiento del sistema mediante diagramas UML de casos de uso y clases.*
- *Aplicar los patrones Singleton, Observer, Factory Method y MVC, justificando su utilidad dentro de la solución propuesta.*
- *Establecer una estructura de ramas Gitflow para organizar el desarrollo y mantener la trazabilidad de los cambios.*

PLANTEAMIENTO DEL PROBLEMA

La gestión de los animales rescatados requiere organización y responsabilidad, ya que es necesario registrar sus datos, conocer su estado de salud, asignar personas encargadas de su cuidado y dar seguimiento a cada caso. Cuando toda esta información se maneja en diferentes documentos o de forma manual, pueden surgir problemas, como la pérdida de datos, la repetición de registros o la dificultad para saber qué atención ha recibido cada animal. Esto también puede retrasar algunas actividades y complicar la comunicación entre las personas que trabajan en el refugio.

Otro aspecto importante es que no todos los animales necesitan los mismos cuidados. Por ejemplo, un perro puede necesitar vacunas, alimentación y una evaluación de su comportamiento, mientras que un animal silvestre puede requerir atención especializada según su especie. Por esta razón, es importante contar con un sistema que permita registrar la información de cada animal de acuerdo con sus necesidades.

Frente a esta situación, se propone desarrollar un sistema que reúna toda la información en un solo lugar, facilite la asignación de voluntarios y veterinarios, y permita actualizar el estado de cada caso desde el rescate hasta una posible adopción. Además, se busca que el sistema genere notificaciones cuando se registre un nuevo rescate, para que las personas responsables puedan conocer la situación y actuar oportunamente. Así, se pretende mejorar la organización del refugio, facilitar el seguimiento de los animales y mantener un historial de las actividades realizadas durante su atención.

ANÁLISIS DEL PROBLEMA

Actores principales

- Ciudadano o reportante: comunica la presencia de un animal que necesita ayuda y proporciona información sobre el lugar y las condiciones del hallazgo.
- Voluntario: atiende los casos asignados, registra las acciones realizadas y actualiza la información correspondiente al rescate.
- Veterinario: revisa el estado de salud, registra diagnósticos y tratamientos y realiza el seguimiento médico del animal.
- Administrador del refugio: administra los usuarios, revisa los casos registrados, asigna responsables y supervisa el proceso de rescate y adopción.
- Adoptante: proporciona sus datos y presenta una solicitud para adoptar un animal disponible.

Requerimientos funcionales

Los requerimientos funcionales describen las acciones que podrá realizar el Sistema de Control de Rescate y Adopción Animal para facilitar la gestión de los animales rescatados.

- RF01. Registrar casos de rescate: El sistema permitirá registrar nuevos casos de animales rescatados, incluyendo la fecha, la ubicación y una descripción de la situación.
- RF02. Clasificar la especie: El sistema permitirá identificar y clasificar a los animales según su especie, como perros, gatos o fauna silvestre.
- RF03. Registrar información del animal: El sistema permitirá almacenar los datos básicos de cada animal, como su nombre, edad aproximada, características físicas, estado de salud y observaciones.
- RF04. Asignar responsables: El administrador podrá asignar voluntarios o veterinarios para atender y realizar el seguimiento de cada caso de rescate.
- RF05. Actualizar el estado del caso: El sistema permitirá actualizar el estado de cada caso según el avance del proceso, utilizando estados como Rescatado, En tratamiento, En recuperación, Disponible para adopción y Adoptado.
- RF06. Registrar tratamientos veterinarios: El veterinario podrá registrar diagnósticos, tratamientos, medicamentos administrados y observaciones sobre la evolución del animal.
- RF07. Enviar notificaciones automáticas: El sistema notificará a los voluntarios y veterinarios suscritos cuando se registre un nuevo caso de rescate o se

produzcan actualizaciones relevantes.

- RF08. Gestionar solicitudes de adopción: El sistema permitirá registrar las solicitudes de las personas interesadas en adoptar un animal y consultar su estado.
- RF09. Aprobar o rechazar adopciones: El administrador podrá revisar las solicitudes de adopción y registrar si han sido aprobadas o rechazadas.
- RF10. Consultar el historial de los casos: El sistema permitirá consultar la información y los cambios realizados en cada caso, incluyendo las asignaciones, los tratamientos y el estado actual del animal.
- RF11. Administrar usuarios: El administrador podrá registrar, consultar, modificar y gestionar los usuarios del sistema, asignándoles los roles correspondientes.
- RF12. Controlar el acceso según el rol: El sistema permitirá que cada usuario acceda únicamente a las funciones autorizadas para su rol, como ciudadano, voluntario, veterinario o administrador.

Requerimientos no funcionales

- Mantenibilidad: el código debe estar organizado en componentes independientes que faciliten las modificaciones.
- Escalabilidad: la arquitectura debe permitir agregar nuevas especies, funciones y usuarios sin tener que reconstruir todo el sistema.
- Seguridad: el acceso a las funciones debe depender del rol de cada usuario.
- Trazabilidad: los cambios en el código deben quedar registrados mediante Git y GitHub.
- Usabilidad: la interfaz debe ser sencilla y permitir encontrar rápidamente la información de cada caso.
- Integridad de la información: los registros deben mantener relaciones coherentes entre animales, casos, responsables y adopciones.

DISEÑO DE LA SOLUCION

Descripción general

el sistema se plantea con una arquitectura basada en el patrón MVC. El usuario interactúa con una interfaz, el controlador recibe las solicitudes y coordina las operaciones, y el modelo contiene las entidades y reglas del negocio.

Para crear expedientes se utilizará Factory Method; para avisar a los responsables se utilizará Observer; y para centralizar el acceso al servicio de gestión de casos se utilizará Singleton.

La información se almacenará en una base de datos. El diseño permitirá registrar los acontecimientos importantes y consultar el historial de cada animal.

Flujo principal del sistema

1. El ciudadano reporta un animal que necesita ayuda.
2. El sistema registra el caso y clasifica la especie.
3. Factory Method crea el expediente correspondiente al tipo de animal.
4. Observer comunica el nuevo caso a los responsables suscritos.
5. El administrador asigna un voluntario o veterinario.
6. El responsable registra las acciones realizadas y actualiza el estado.
7. Cuando el animal cumple las condiciones necesarias, se registra su disponibilidad para adopción.
8. El administrador revisa la solicitud y registra el resultado del proceso.

Los cambios de estado deben estar sujetos a las reglas del refugio. Por ejemplo, un animal no debería pasar directamente a Adoptado sin que exista un registro de adopción aprobado.

APLICACIÓN DE SISTEMAS DE DISEÑO

Singleton

Este patrón garantiza una única instancia compartida de un componente durante el ciclo de vida de la aplicación.En el proyecto se aplicará al servicio central de gestión de casos, denominado GestorCasos. Su responsabilidad será coordinar las operaciones principales relacionadas con los casos de rescate.

Justificación técnica: permite centralizar el acceso a las operaciones de gestión y evita crear varias instancias innecesarias del mismo servicio. Cabe recalcar que Singleton no se utiliza para todas las clases.

Observer

Este patrón permite que un objeto notifique automáticamente a otros objetos cuando ocurre un evento.

En el sistema, cuando se registra un nuevo caso, el servicio de notificaciones comunica el acontecimiento a los voluntarios o veterinarios que estén suscritos a los avisos correspondientes.

Justificación técnica: reduce el acoplamiento entre el registro de casos y el mecanismo de notificaciones. En el futuro, se podrán incorporar avisos por correo electrónico o notificaciones internas sin modificar toda la lógica de rescate.

Factor method

Este patrón permite crear objetos sin que el código principal tenga que conocer todos los detalles de su construcción.

En el proyecto se utilizará para generar expedientes específicos de perros, gatos y fauna silvestre. Cada tipo de expediente podrá incluir información particular, manteniendo una estructura común.

Justificación técnica: facilita incorporar nuevas especies y evita concentrar todas las condiciones de creación en una sola clase.

MVC

MVC separa la aplicación en tres componentes:

- Model: contiene las entidades, los datos y las reglas del negocio.
- View: presenta los formularios, listados, detalles de los animales y estados de los casos.
- Controller: recibe las acciones del usuario, valida las solicitudes y coordina las operaciones del modelo.

Justificación técnica: esta separación facilita el mantenimiento, las pruebas y la modificación de la interfaz sin tener que reescribir las reglas principales del sistema.

RESULTADOS

Con el diseño de este proyecto se espera tener una estructura organizada del sistema, en la que se puedan identificar las principales funciones del refugio y la forma en que se relacionan entre sí. Esto permitirá comprender mejor cómo se manejará la información de los animales rescatados y cómo se llevará el seguimiento de cada caso.

Para ello, los diagramas UML ayudarán a representar de manera clara el funcionamiento del sistema, las personas que lo utilizarán y las clases que formarán parte de su estructura. Asimismo, los patrones de diseño permitirán organizar mejor el código, distribuir las responsabilidades y facilitar funciones como el envío de notificaciones y la creación de expedientes según el tipo de animal. También se utilizará Gitflow para organizar los cambios realizados en el proyecto y llevar un registro de sus avances.

En general, este diseño servirá como punto de partida para desarrollar una aplicación que ayude a mejorar la organización y el control de los animales rescatados. Sin embargo, es importante tener en cuenta que, por el momento, se trata de una propuesta de diseño. La programación del sistema, las pruebas de funcionamiento y la conexión con una base de datos se realizarán en las siguientes etapas del proyecto.

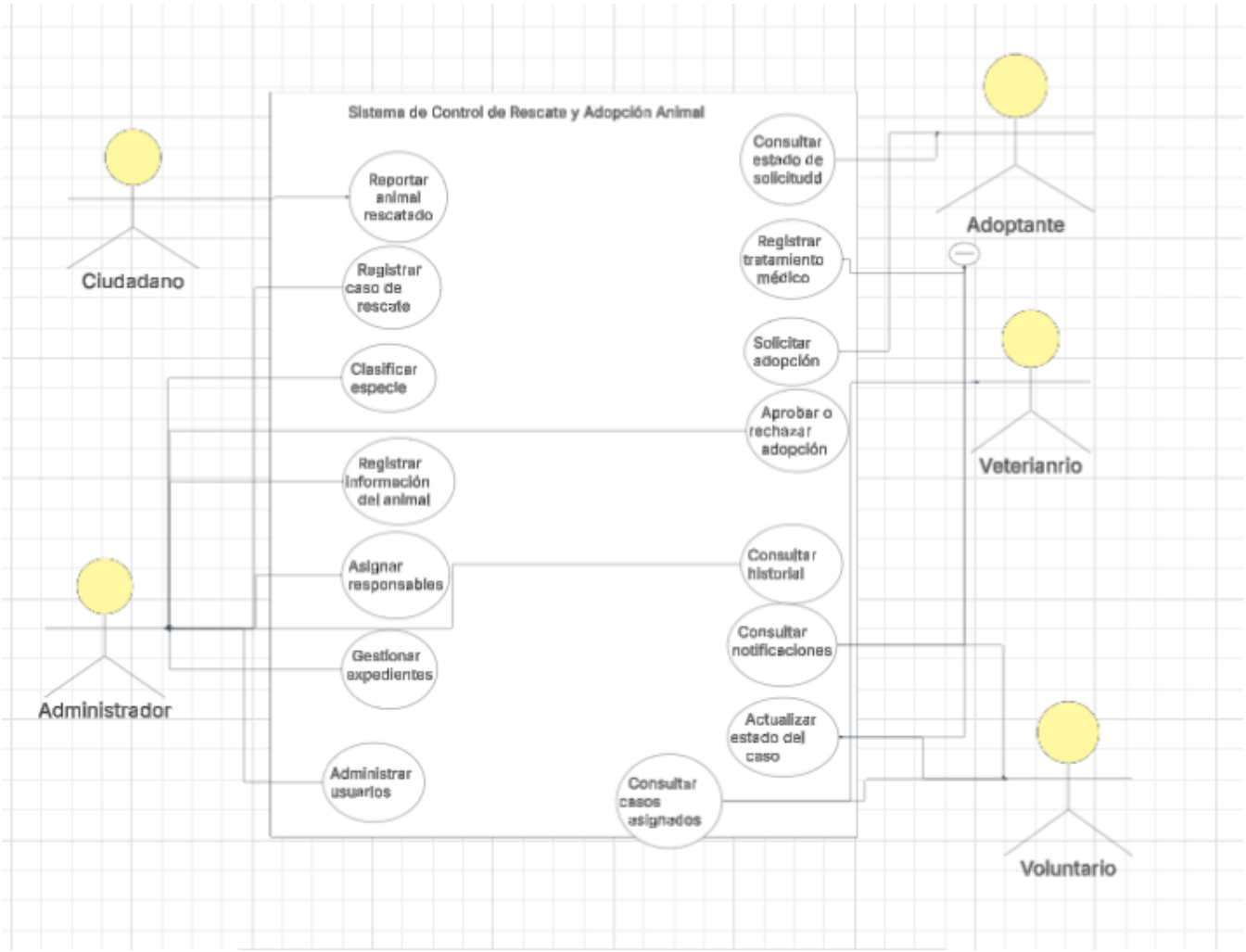
CONCLUSIONES

El diseño del Sistema de Control de Rescate y Adopción Animal permite tener una mejor organización de la información de los animales rescatados, sus responsables, los tratamientos que reciben y su proceso de adopción. De esta manera, se facilita el seguimiento de cada caso y se evita que los datos importantes se pierdan o se encuentren dispersos en diferentes documentos.

El uso de los patrones de diseño Singleton, Observer, Factory Method y MVC, junto con los principios SOLID y la metodología Gitflow, ayuda a plantear un sistema más organizado y fácil de mantener. Además, trabajar con estas herramientas permite comprender mejor cómo se estructura un proyecto de software y cómo se pueden realizar futuras mejoras sin afectar todo el sistema.

DIAGRAMAS DEL SISTEMA

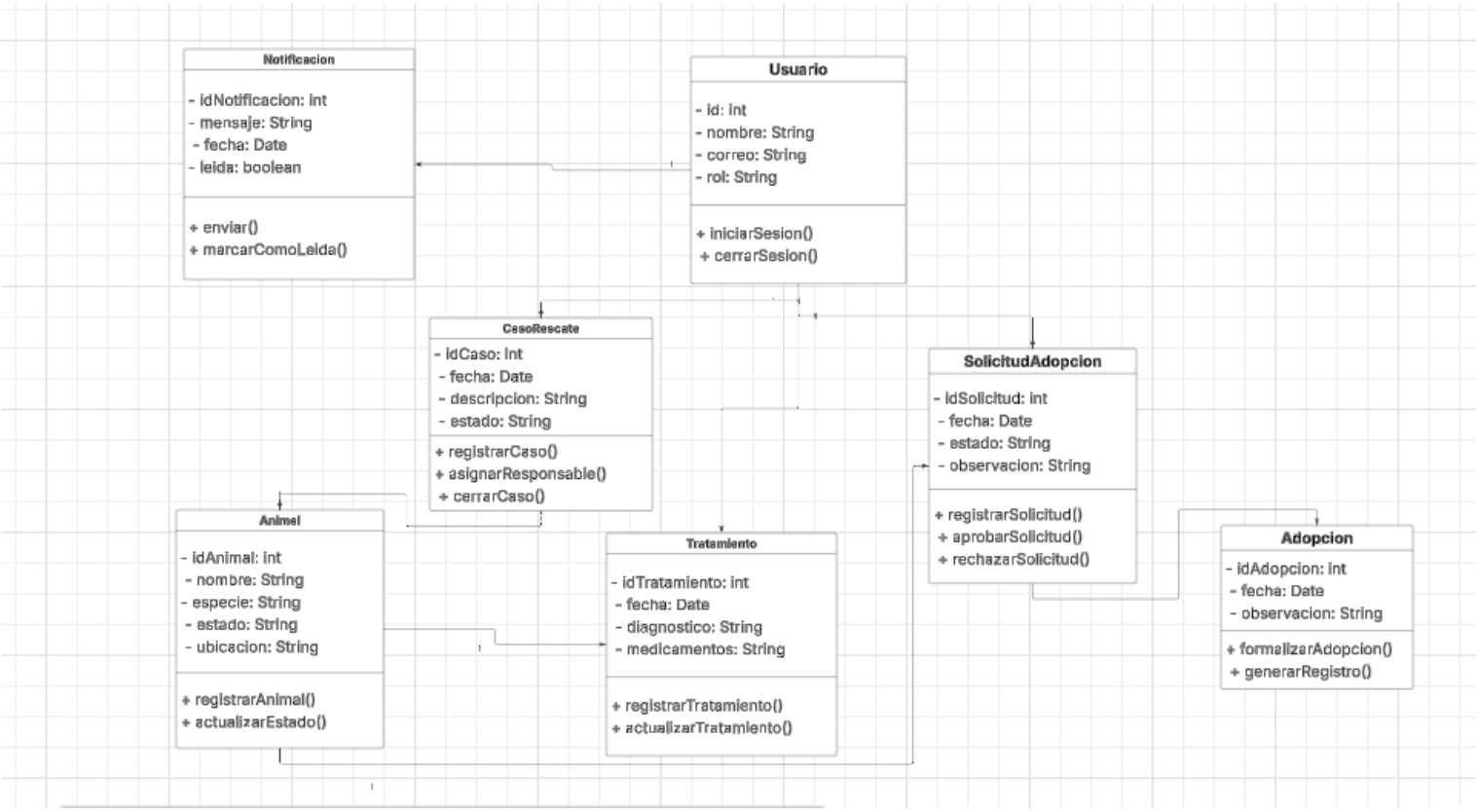
Diagrama 1. Casos de uso



Enlace

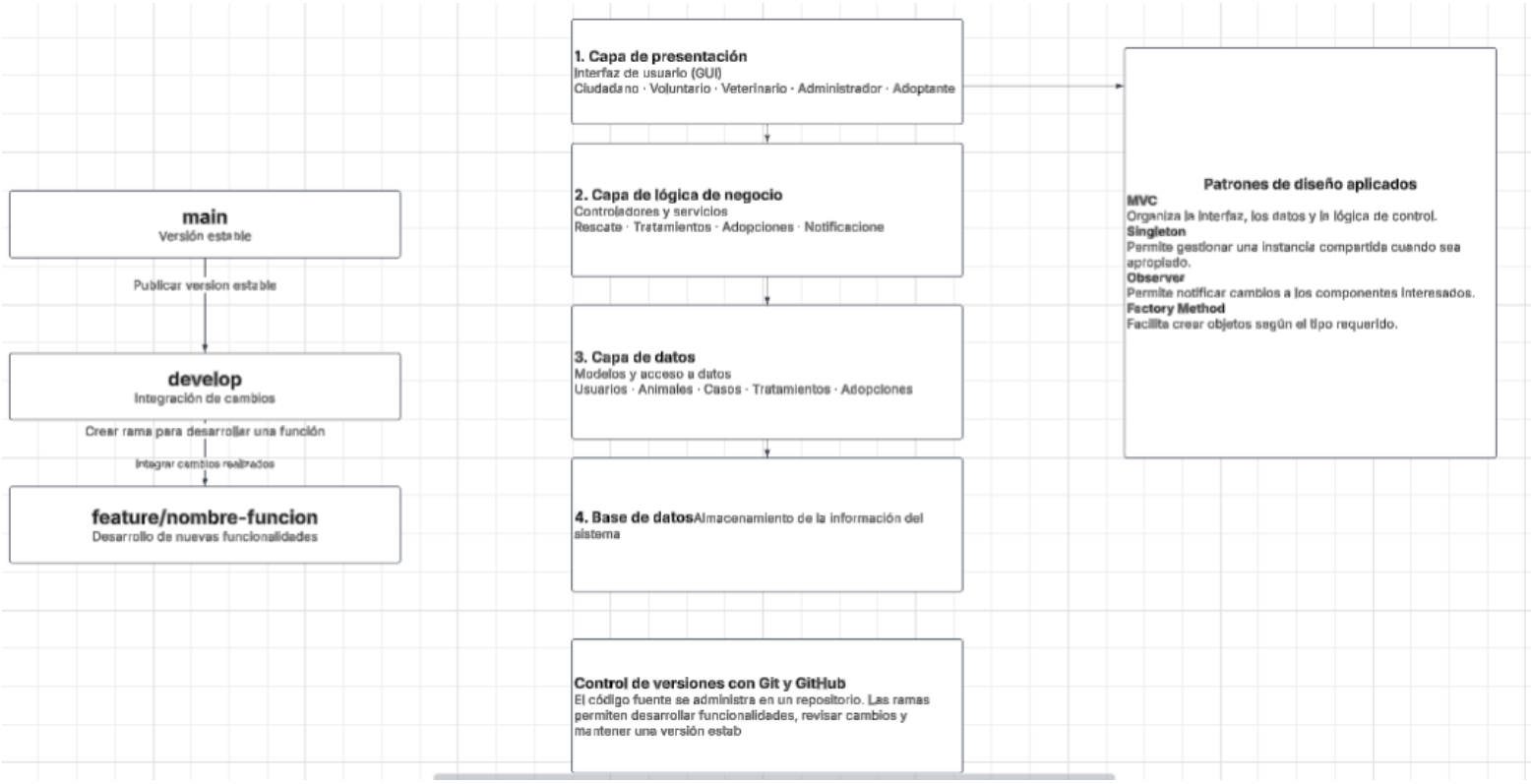
https://lucid.app/lucidchart/12189baa-0a67-418f-8900-6f2e60b13740/edit?viewport_loc=-9688%2C-1907%2C12655%2C5123%2C0_0&invitationId=inv_bb35b50d-f406-4b83-b311-049770afa3aa

Diagrama 2. Clase UML



Enlace
https://lucid.app/lucidchart/3c94fa33-6e36-4c0b-bd29-9144ba351967/edit?viewport_loc=-3368%2C-828%2C5464%2C2212%2C0_0&invitationId=inv_ce98d4f1-9933-440a-b013-86e898ddf1dc

Diagrama3. Arquitectura general y gitflow



Enlace

https://lucid.app/lucidchart/1ce1a5c7-ad48-4218-99fa-70270605fa25/edit?viewport_loc=308%2C952%2C683%2C277%2C0_0&invitationId=inv_a1bf7a70-6711-4b39-96c5-ae64b28fd658

Referencias Bibliograficas

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Git. (s. f.). Git documentation. https://git-scm.com/doc

GitHub. (s. f.). About branches. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches